Java Class 10 — Encapsulation & Access Modifiers

Since students have already learned Class, Object, Constructor, Parameterized Constructor, and **`this`**, Class 10 should now teach them how to protect object data.

1. Learning Objectives

By the end of this class, students should be able to:

 Understand access modifiers

 Understand `public`, `private`, `protected`, and default access

 Understand data hiding

 Understand encapsulation

 Make variables `private`

 Create getters and setters

 Access private data through methods

 Add validation using setters

 Understand why direct access to variables can be dangerous

 Create properly encapsulated classes

 Understand the difference between getter and setter

 Apply encapsulation to real-world examples

2. Quick Revision

Start with the previous class.

```
class Student {

    String name;
    int age;

    Student(String name, int age) {
```

```
        this.name = name;
        this.age = age;
    }

    void display() {
        System.out.println(name);
        System.out.println(age);
    }
}
```

Ask:

> Can someone outside the class directly change `age`?

Yes.

```
Student s = new Student("Rahul", 20);

s.age = -50;
```

This is a problem.

A student cannot logically have an age of `-50`.

So we need a way to protect our data.

This leads to encapsulation.

3. What is Encapsulation?

Encapsulation is the process of wrapping data and methods together inside a class and controlling access to that data.

In simple words:

> Keep data protected and provide controlled ways to access or modify it.

Example:

```
class Student {

    private String name;
    private int age;
```

```
    public void setAge(int age) {

        if (age > 0) {
            this.age = age;
        }
    }

    public int getAge() {
        return age;
    }
}
```

Here:

```
private data
     ↓
Getter / Setter
     ↓
Controlled access
```

4. Why Do We Need Encapsulation?

Without encapsulation:

```
class BankAccount {

    double balance;
}
```

Anyone can do:

```
BankAccount account = new BankAccount();

account.balance = -50000;
```

There is no protection.

With encapsulation:

```
class BankAccount {
```

```
    private double balance;
}
```

Now:

```
account.balance = -50000;
```

will give an error.

The data can only be accessed through methods that we provide.

5. Access Modifiers

Java provides access modifiers to control accessibility.

The four main access levels are:

```
public
private
protected
default
```

6. `public`

A `public` member can generally be accessed from anywhere the class itself is accessible.

Example:

```
class Student {

    public String name;
}
```

We can do:

```
Student s = new Student();

s.name = "Rahul";
```

```
System.out.println(s.name);
```

7. `private`

A `private` member can be accessed only inside the same class.

Example:

```
class Student {

    private int age;
}
```

This is not allowed:

```
Student s = new Student();

s.age = 20;
```

The compiler gives an access error because `age` is private.

8. Why `private` Is Important

Consider a bank account.

We don't want anyone to directly modify:

```
balance
```

Instead of:

```
account.balance = -100000;
```

we want:

```
account.deposit(5000);
```

or:

```
account.withdraw(2000);
```

The class controls what is allowed.

9. Getter

A getter is a method used to read/access a private variable.

Example:

```
class Student {

    private int age;

    public int getAge() {
        return age;
    }
}
```

Usage:

```
Student s = new Student();

System.out.println(s.getAge());
```

Think:

```
getAge()
   ↓
GET / READ data
```

10. Setter

A setter is a method used to modify/update a private variable.

Example:

```
class Student {

    private int age;

    public void setAge(int age) {
        this.age = age;
    }
}
```

Usage:

```
Student s = new Student();

s.setAge(20);
```

Think:

```
setAge()
   ↓
SET / CHANGE data
```

11. Getter + Setter Together

```
class Student {

    private String name;
    private int age;

    public String getName() {
        return name;
    }

    public void setName(String name) {
```

```
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }
}
```

Usage:

```
Student s = new Student();

s.setName("Rahul");
s.setAge(20);

System.out.println(s.getName());
System.out.println(s.getAge());
```

Output:

```
Rahul
20
```

12. Why Not Just Make Everything `public`?

Ask students:

Suppose:

```
class BankAccount {

    public double balance;
}
```

Then:

```
account.balance = -5000;
```

The class has no control.

But:

```
class BankAccount {

    private double balance;

    public void setBalance(double balance) {

        if (balance >= 0) {
            this.balance = balance;
        }
    }
}
```

Now invalid values can be rejected.

This is the real benefit of encapsulation.

13. Encapsulation with Validation

This is an important practical example.

```
class Student {

    private int age;

    public void setAge(int age) {

        if (age >= 5 && age <= 100) {
            this.age = age;
        } else {
            System.out.println("Invalid age");
        }
    }

    public int getAge() {
        return age;
```

```
    }
}
```

Now:

```
Student s = new Student();

s.setAge(20);
System.out.println(s.getAge());
```

Output:

```
20
```

But:

```
s.setAge(-10);
```

Output:

```
Invalid age
```

14. Real-World Example — Bank Account

This is an excellent example to teach encapsulation.

```
class BankAccount {

    private String accountHolder;
    private double balance;

    BankAccount(String accountHolder, double balance) {

        this.accountHolder = accountHolder;

        if (balance >= 0) {
            this.balance = balance;
        }
```

```
    }

    public String getAccountHolder() {
        return accountHolder;
    }

    public double getBalance() {
        return balance;
    }

    public void deposit(double amount) {

        if (amount > 0) {
            balance = balance + amount;
        }
    }

    public void withdraw(double amount) {

        if (amount > 0 && amount <= balance) {
            balance = balance - amount;
        } else {
            System.out.println("Invalid withdrawal");
        }
    }
}
```

Usage:

```
BankAccount account =
    new BankAccount("Rahul", 50000);

account.deposit(10000);

account.withdraw(5000);

System.out.println(account.getBalance());
```

Output:

```
55000.0
```

Notice:

We cannot directly do:

```
account.balance = -50000;
```

because `balance` is private.

15. Data Hiding

Data hiding means preventing direct access to internal data.

Example:

```
class Employee {

    private double salary;
}
```

Outside code cannot directly access:

```
employee.salary;
```

Instead, controlled methods can be provided.

```
public double getSalary() {
    return salary;
}
```

16. Encapsulation vs Data Hiding

Students often treat them as exactly the same.

Data Hiding

Focuses on restricting access to data.

Example:

```
private int age;
```

Encapsulation

Focuses on bundling data and methods together and controlling access.

Example:

```
class Student {

    private int age;

    public void setAge(int age) {
        // validation
    }

    public int getAge() {
        return age;
    }
}
```

So:

> **`private`** helps achieve data hiding, while getters/setters and controlled methods are part of encapsulation.

17. `protected`

`protected` provides access within the same package and also to subclasses under Java's access rules.

Example:

```
class Parent {

    protected int value = 10;
}
```

It becomes particularly important when students learn inheritance.

For now, tell students:

> `protected` is mainly useful when working with inheritance and packages.

18. Default Access

If we don't specify an access modifier:

```
class Student {

    int age;
}
```

then it has default/package-private access.

It can be accessed by classes in the same package.

Example:

```
class Student {

    int age;
}
```

No modifier is written.

19. Access Modifier Summary

ModifierSame ClassSame PackageSubclassOther Package`private`YesNoNoNodefaultYesYesVia package accessNo`protected`YesYesYesYes, subject to subclass access rules`public`YesYesYesYes

For beginner students, the easiest memory trick is:

```
private
   ↓
Only my class

default
   ↓
My package

protected
```

```
   ↓
My package + child classes

public
   ↓
Everywhere accessible
```

20. Complete Encapsulation Example

```
class Employee {

    private String name;
    private double salary;

    Employee(String name, double salary) {

        this.name = name;

        if (salary >= 0) {
            this.salary = salary;
        }
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {

        if (salary >= 0) {
            this.salary = salary;
        } else {
            System.out.println("Salary cannot be negative");
        }
    }
```

```
}
```

Main:

```
public class Main {

    public static void main(String[] args) {

        Employee e1 =
            new Employee("Rahul", 50000);

        System.out.println(e1.getName());
        System.out.println(e1.getSalary());

        e1.setSalary(60000);

        System.out.println(e1.getSalary());
    }
}
```

21. Important Point: Getter Doesn't Always Mean Setter

You don't have to provide both.

For example:

```
class Employee {

    private double salary;

    public double getSalary() {
        return salary;
    }
}
```

This makes salary read-only from outside the class.

Outside code can:

```
System.out.println(e.getSalary());
```

But cannot:

```
e.setSalary(100000);
```

because there is no setter.

This is useful when you want a value to be readable but not directly changeable.

22. Read-Only Property

Example:

```
class Student {

    private final int rollNo;

    Student(int rollNo) {
        this.rollNo = rollNo;
    }

    public int getRollNo() {
        return rollNo;
    }
}
```

There is a getter but no setter.

Students will later learn why `final` is useful here.

23. Why Use Methods Instead of Direct Variables?

Compare:

Without encapsulation

```
account.balance = -5000;
```

With encapsulation

```
account.withdraw(5000);
```

The second approach is better because the object controls its own state.

This is a key OOP idea:

> An object should control how its data is changed.

24. Common Student Mistakes

Mistake 1

Trying to access a private variable directly:

```
s.age = 20;
```

If `age` is private, this is not allowed.

Use:

```
s.setAge(20);
```

Mistake 2

Forgetting `this`

Wrong:

```
public void setName(String name) {
    name = name;
}
```

Correct:

```
public void setName(String name) {
    this.name = name;
}
```

Mistake 3

Writing getter incorrectly

Wrong:

```
public int getAge() {
    age;
}
```

Correct:

```
public int getAge() {
    return age;
}
```

Mistake 4

Thinking setter must return a value

Normally:

```
public void setAge(int age) {
    this.age = age;
}
```

A setter generally returns `void`.

25. Practice Questions

Level 1

Q1 — Student

Create:

```
Student
----------------
private name
private age
```

Create:

```
getName()
setName()

getAge()
setAge()
```

Q2 — Employee

Create an `Employee` class:

```
name
salary
department
```

Make all variables private and create getters and setters.

Level 2

Q3 — Validation

Create a `BankAccount`.

Requirements:

- `balance` must be private.
- Deposit amount must be greater than 0.
- Withdrawal cannot exceed balance.
- Provide a getter for balance.

Q4 — Product

Create:

```
Product
----------------
```

```
private name
private price
private quantity
```

Rules:

```
price >= 0
quantity >= 0
```

Implement getters and setters with validation.

Level 3

Q5 — College Student

Create:

```
Student
----------------
private name
private rollNo
private marks
```

Rules:

```
marks must be between 0 and 100
```

Create:

```
getName()
setName()

getRollNo()
setRollNo()

getMarks()
setMarks()
```

26. Class Assignment

Create a class:

```
BankAccount
```

Variables:

```
private accountNumber
private accountHolder
private balance
```

Constructor:

```
BankAccount(accountNumber, accountHolder, balance)
```

Methods:

```
deposit()
withdraw()
getBalance()
displayAccountDetails()
```

Rules:

```
Deposit > 0
Withdrawal > 0
Withdrawal <= balance
Balance cannot be negative
```

The student should not be able to directly modify **`balance`**.

What Students Should Know After Class 10

By the end of this class, students should be able to:

□ Define encapsulation

□ Explain data hiding

□ Understand why variables are made `private`

□ Understand access modifiers

□ Understand `public`

□ Understand `private`

□ Understand `protected`

□ Understand default/package-private access

□ Create private instance variables

□ Create getters

□ Create setters

□ Use `this` inside setters

□ Add validation inside setters

□ Create read-only properties

□ Explain getter vs setter

□ Understand why direct data access can be dangerous

□ Build an encapsulated class

□ Apply encapsulation to real-world problems

Next Class — Class 11: Inheritance

The next class should begin:

Inheritance → **`extends`** → Parent/Child class → Code reuse → **`super`** keyword → Parent constructor → Types of inheritance → Method inheritance → Real-world examples.

The key progression will be:

```
Class & Object
      ↓
Constructor
      ↓
this
      ↓
Access Modifiers
```

```
      ↓
Encapsulation
      ↓
Getter / Setter
      ↓
Inheritance
      ↓
Polymorphism
      ↓
Abstraction
      ↓
Interface
```

-